

Machine Learning

10. Neural networks and deep learning

Yannick Le Cacheux

CentraleSupélec - Université Paris Saclay

September 2024

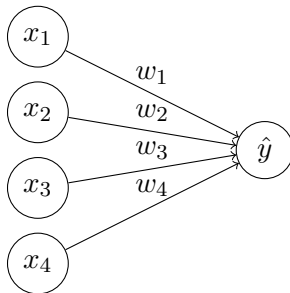
Outline

- 1 Artificial neural networks
- 2 Back-propagation
- 3 Variational auto-encoders
 - Limits of PCA
 - Alternative view

Logistic regression

- In logistic regression, for input vector $\mathbf{x} \in \mathbb{R}^D$, output probability $\hat{y}$ is estimated with

$$\hat{y} = \sigma(w_1x_1 + w_2x_2 + \cdots + w_Dx_D) = \sigma(\mathbf{w}^\top \mathbf{x})$$



$$\mathbf{x} \in \mathbb{R}^D \quad \hat{y} = \sigma(\mathbf{w}^\top \mathbf{x}) \in [0, 1]$$

Artificial neuron

- This resembles a single neuron: the inputs w_d represent the *dendrites*, and the output $\hat{y}$ the *axon*.
- If enough “stimulation” arrives from the dendrites (*i.e.* $\mathbf{w}^\top \mathbf{x} \gg 0$), the neuron fires: $\hat{y} \approx 1$
- Otherwise, if $\mathbf{w}^\top \mathbf{x} \ll 0$, nothing happens: $\hat{y} \approx 0$

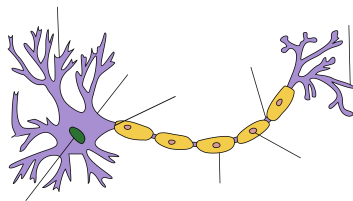
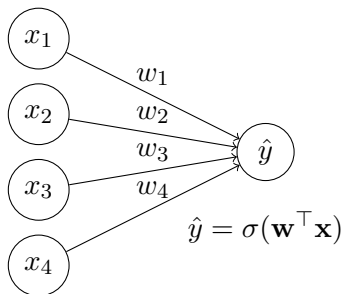
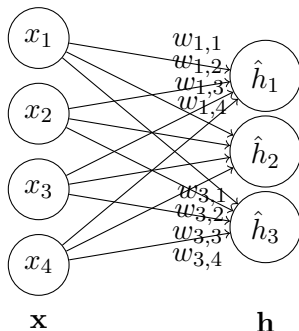


Figure: Representation of a human neuron, from Wikipedia

Multi-output logistic regression

- We can do multiple logistic regression at the same time to have H outputs $\mathbf{h} = (\hat{h}_1, \dots, \hat{h}_H)^\top$



$$\begin{aligned}\hat{h}_1 &= \sigma(w_{1,1}x_1 + \dots + w_{1,D}x_D) \\ &= \sigma(\mathbf{w}_1^\top \mathbf{x})\end{aligned}$$

$$\hat{h}_2 = \sigma(\mathbf{w}_2^\top \mathbf{x})$$

...

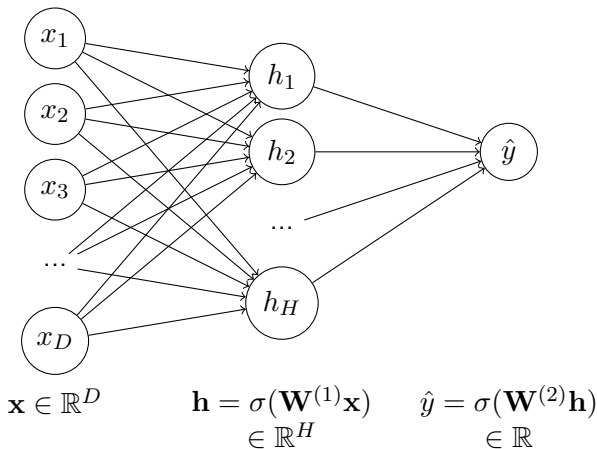
$$\hat{h}_H = \sigma(\mathbf{w}_H^\top \mathbf{x})$$

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x})$$

- We now have multiple neurons

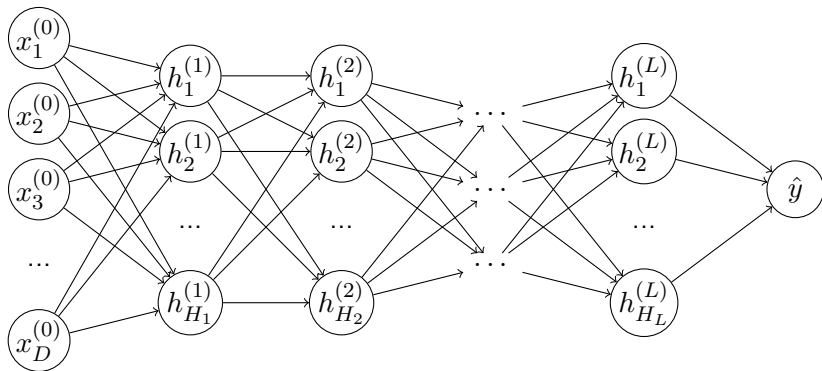
Artificial neural network

- In the brain, the outputs of neurons are the inputs to other neurons
- This is equivalent to using $\mathbf{h}$ as the features to other model(s)



Deep learning

- We can stack even more layers, to obtain a *deep* neural network with L hidden layers $\mathbf{h}^{(1)}, \dots, \mathbf{h}^{(L)}$



$$\mathbf{x}^{(0)} \in \mathbb{R}^D \quad \mathbf{h}^{(1)} \in \mathbb{R}^{H_1} \quad \mathbf{h}^{(2)} \in \mathbb{R}^{H_2} \quad \mathbf{h}^{(L)} \in \mathbb{R}^{H_L} \quad \hat{y} \in [0, 1]$$

$$= \sigma(\mathbf{W}^{(1)}\mathbf{x}) = \sigma(\mathbf{W}^{(2)}\mathbf{h}^{(1)}) = \sigma(\mathbf{W}^{(L)}\mathbf{h}^{(L-1)}) \quad \sigma(\mathbf{W}^{(L+1)}\mathbf{h}^{(L)})$$

- In general $\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)}) \in \mathbb{R}^{H_l}$

Training a deep neural net

- How do we find the appropriate values for all P parameters $\boldsymbol{\theta} = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L+1)}\}_{\text{flattened}} \quad (\in \mathbb{R}^P)$ of the network?
- Given training dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$ and a loss function $J_{\mathcal{D}}(\boldsymbol{\theta})$, for example

$$J_{\mathcal{D}}(\boldsymbol{\theta}) = - \sum_n (y_n \log(\hat{y}_n) + (1 - y_n) \log(1 - \hat{y}_n))$$

- We can apply gradient descent: at time step t ,

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \alpha \frac{\partial J_{\mathcal{D}}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_{t-1}}$$

Computing the gradients

- Gradient for parameter θ_i can be numerically estimated with finite differences method: apply a small change ϵ in direction $\mathbf{e}_i$ of θ_i , and measure the impact on the loss:

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_i} \approx \frac{J(\boldsymbol{\theta} + \epsilon \mathbf{e}_i) - J(\boldsymbol{\theta})}{\epsilon}$$

- This can be very computationally intensive: to obtain the gradient of a single parameter θ_i , we need to apply a small change ϵ and propagate the impact forward in the whole network until we can measure the impact on the loss J . Then repeat these operations on every single parameter.
 - ▶ And we may have many layers, many parameters and many training samples.
- This is not efficient at scale.

Back-propagation

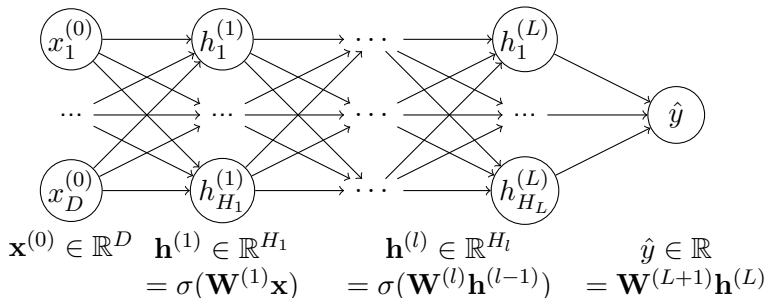
- Main idea: use dynamic programming and the chain rule of derivatives to avoid redundant computations in gradient estimation.

$$\underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{x}}}_{\in \mathbb{R}^{N \times L}} = \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{y}}}_{\in \mathbb{R}^{N \times M}} \cdot \underbrace{\frac{\partial \mathbf{y}}{\partial \mathbf{x}}}_{\in \mathbb{R}^{M \times L}}$$

- (*Cf. next section*)

Neural network for regression

- For regression, the idea is exactly the same: we compute intermediate features as hidden layers $\mathbf{h}^{(l)}$ and find suitable parameters with gradient descent, with two changes:
 - We remove the output sigmoid σ so that $\hat{y} = \mathbf{W}^{(L+1)}\mathbf{h}^{(L)}$ can have values in $\mathbb{R}$
 - We use a suitable loss function J (e.g. an MSE) to train the network



- Should we keep the intermediate sigmoids $\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)})$?

Activation function

- In practice, we often use *rectified linear units* (ReLU) as *activation* functions instead of the sigmoid σ for intermediate layers¹:

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- Is it necessary to use an activation function?

¹This is to avoid a problem called *vanishing gradients*; out of scope for now.

Activation function

- In practice, we often use *rectified linear units* (ReLU) as *activation* functions instead of the sigmoid σ for intermediate layers¹:

$$\text{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- Is it necessary to use an activation function?

Without activation functions:

- Yes: a linear combination of linear combinations is a linear combination.

$$\hat{y} = \mathbf{W}^{(L+1)} \mathbf{h}^{(L)} \quad \text{and} \quad \mathbf{h}^{(L)} = \mathbf{W}^{(L)} \mathbf{h}^{(L-1)}$$

- Without non-linear activation functions, our network is equivalent to a network with no hidden layer: we would be doing linear regression (with unnecessary complications)

$$\begin{aligned} \text{so } \hat{y} &= \mathbf{W}^{(L+1)} \mathbf{W}^{(L)} \mathbf{h}^{(L-1)} \\ &= \mathbf{W}^{(L+1)} \mathbf{W}^{(L)} \mathbf{W}^{(L-1)} \mathbf{h}^{(L-2)} \\ &= \dots \\ &= \mathbf{W}^{(L+1)} \mathbf{W}^{(L)} \dots \mathbf{W}^{(1)} \mathbf{x} \\ &= \mathbf{W}_{\text{eq}} \mathbf{x} \end{aligned}$$

¹This is to avoid a problem called *vanishing gradients*; out of scope for now.

Deep learning is a whole field

There are many additional things that we would need to cover:

- Different architectures: number of hidden layers, dimensions of the hidden layers...
- But also different ways to organize and connect neurons: convolutional neural networks (CNN), recurrent neural networks, transformers...
- Different ways to efficiently train a network: stochastic gradient descent, momentum, batch norm...
- Different ways to limit overfitting: early stopping, weight penalty, drop-out...
- ...

But all of this is out of scope for now.

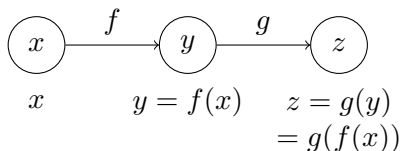
¹Cf. I. Goodfellow et al., *Deep learning* for a good introduction, and S. Prince, *Understanding deep learning* for coverage of more recent topics (transformers, diffusion models etc)

Outline

- 1 Artificial neural networks
- 2 Back-propagation**
- 3 Variational auto-encoders
 - Limits of PCA
 - Alternative view

Chain rule of derivatives

- Let us consider values $x, y, z \in \mathbb{R}$, and functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$:



- Chain rule of derivatives²:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

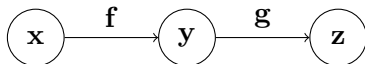
- (French high-school version):

$$\underbrace{g(f(x))'}_{\frac{\partial z}{\partial x}} = \underbrace{f'(x)}_{\frac{\partial y}{\partial x}} \cdot \underbrace{g'(f(x))}_{\frac{\partial z}{\partial y}}$$

²Assuming derivatives exist etc etc

Chain rule of derivatives

- Given $\mathbf{x} \in \mathbb{R}^L$, $\mathbf{y} \in \mathbb{R}^M$, $\mathbf{z} \in \mathbb{R}^N$ and $\mathbf{f} : \mathbb{R}^L \rightarrow \mathbb{R}^M$, $\mathbf{g} : \mathbb{R}^M \rightarrow \mathbb{R}^N$:



- Chain rule of derivatives, vector version:

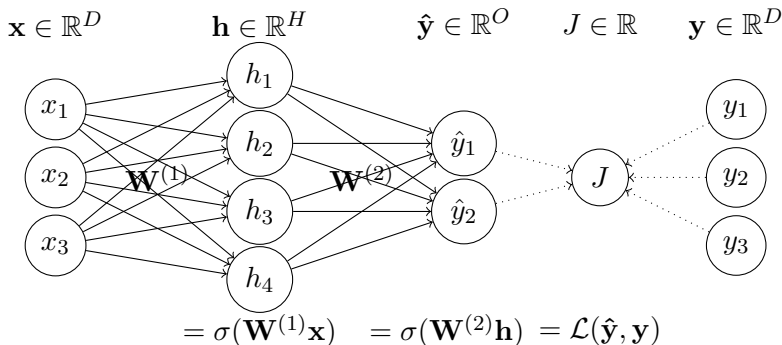
$$\underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{x}}}_{\in \mathbb{R}^{N \times L}} = \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{y}}}_{\in \mathbb{R}^{N \times M}} \cdot \underbrace{\frac{\partial \mathbf{y}}{\partial \mathbf{x}}}_{\in \mathbb{R}^{M \times L}}$$

where “ $\cdot$ ” is now the matrix product³

³or tensor product in an even more general case

Application to deep learning

- Let us consider the following neural network trained with loss J
 - How many outputs O does it have? Hidden layers L ? Parameters P ?...



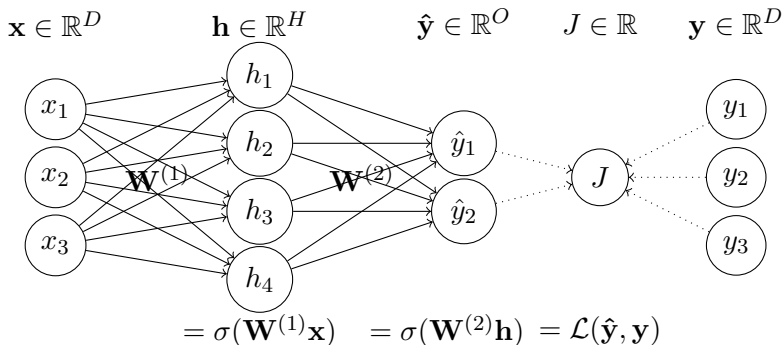
We want the gradient $\frac{\partial J}{\partial \theta}$ of all the P parameters θ_p in $\theta = \{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}\}$

- So we want $\frac{\partial J}{\partial \mathbf{W}^{(1)}}$ and $\frac{\partial J}{\partial \mathbf{W}^{(2)}}$

³ σ can be any (non linear) activation function

Application to deep learning

- Let us consider the following neural network trained with loss J
 - $P = H \times D + O \times H$. Here $L = 1$, $D = 4$, $H = 3$, $O = 2$.



We want the gradient $\frac{\partial J}{\partial \theta}$ of all the P parameters θ_p in $\theta = \{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}\}$

- So we want $\frac{\partial J}{\partial \mathbf{W}^{(1)}}$ and $\frac{\partial J}{\partial \mathbf{W}^{(2)}}$

³ σ can be any (non linear) activation function

Gradient of $\mathbf{W}^{(2)}$

- $\mathbf{W}^{(2)}$: this is a matrix of size $O \times H$, we can “flatten” it to a vector $\mathbf{w}^{(2)}$ of size $S_2 = O \cdot H$. Then

$$\frac{\partial J}{\partial \mathbf{w}^{(2)}} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{w}^{(2)}}$$

- We can estimate $\frac{\partial J}{\partial \hat{\mathbf{y}}}$ and $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{w}^{(2)}}$ with the finite differences method:
 apply small perturbations on $\mathbf{w}^{(2)}$ and measure the impact on $\hat{\mathbf{y}}$, apply small perturbations on $\hat{\mathbf{y}}$ on measure the impact on J .
- Note: as the other gradients, $\frac{\partial J}{\partial \hat{\mathbf{y}}}$ depends on the choice of the loss function J . For the MSE $J = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$ we have $\frac{\partial J}{\partial \hat{\mathbf{y}}} = 2(\hat{\mathbf{y}} - \mathbf{y})$. For intermediate gradients that rely on matrix multiplication, we can similarly find more efficient computations than finite differences method. But in general, we may not always have analytical solutions.

Gradient of $\mathbf{W}^{(1)}$

- We similarly flatten $\mathbf{W}^{(1)} \in \mathbb{R}^{H \times D}$ to $\mathbf{w}^{(1)} \in \mathbb{R}^{H \cdot D}$, and then

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{w}^{(1)}} &= \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \underbrace{\left(\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{h}} \right)}_{\frac{\partial \mathbf{w}^{(2)}}{\partial \mathbf{h}}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{w}^{(1)}} \\ &= \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \left(\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{w}^{(2)}} \cdot \frac{\partial \mathbf{w}^{(2)}}{\partial \mathbf{h}} \right) \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{w}^{(1)}} \end{aligned}$$

- We have already computed the first 2 factors $\frac{\partial J}{\partial \hat{\mathbf{y}}}$ and $\frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{w}^{(2)}}$ when computing the gradient of $\mathbf{W}^{(2)}$
- All that is left to do is compute the last 2 factors $\frac{\partial \mathbf{w}^{(2)}}{\partial \mathbf{h}}$ and $\frac{\partial \mathbf{h}}{\partial \mathbf{w}^{(1)}}$, again with a numerical estimation
- This is efficient: we do not need to propagate forward from $\mathbf{w}^{(1)}$ along the whole network until we reach J

Gradient of $\mathbf{W}^{(l)}$

- Generalization: with L hidden layers $\mathbf{h}^{(1)}, \dots, \mathbf{h}^{(L)}$ with associated weights $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ and $\mathbf{W}^{(L+1)}$ (output weights)
- We first compute $\frac{\partial J}{\partial \mathbf{w}^{(L+1)}}$, then $\frac{\partial J}{\partial \mathbf{w}^{(L)}}$, $\frac{\partial J}{\partial \mathbf{w}^{(L-1)}}$ etc, using

$$\frac{\partial J}{\partial \mathbf{w}^{(l)}} = \underbrace{\frac{\partial J}{\partial \mathbf{w}^{(l+1)}}}_{\text{already computed}} \cdot \underbrace{\frac{\partial \mathbf{w}^{(l+1)}}{\partial \mathbf{h}^{(l)}} \cdot \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{w}^{(l)}}}_{\text{left to estimate}}$$

- We *propagate* the gradients backwards, hence the name *back-propagation*

Back-propagation: recap

Back-propagation

Back-propagation is a method that lets us efficiently estimate the gradients of all parameters of a (deep) neural network, by propagating already computed gradients backwards in the network to avoid redundant computations.

- We can then find good parameters $\hat{\theta}$ by applying a simple gradient descent:

$$\theta_t = \theta_{t-1} - \alpha \frac{\partial J_{\mathcal{D}}(\theta)}{\partial \theta_{t-1}}$$

or more complex gradient-based methods (Adam, etc)

Outline

- 1 Artificial neural networks
- 2 Back-propagation
- 3 Variational auto-encoders
 - Limits of PCA
 - Alternative view

Data generation: examples from Lab 3



Figure: Examples of handwritten images of “5”



Figure: Samples generated from 8 principal components

- Are generated samples indistinguishable from real samples?

Trying

- 8 principal components only explain 50% of the variance
- Maybe we can sample from more principal components?

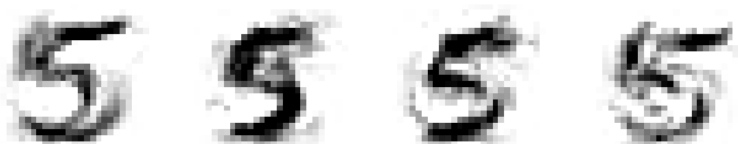
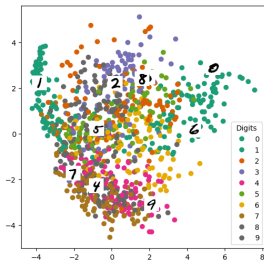


Figure: Samples generated from 784 principal components, explaining 100% of the variance

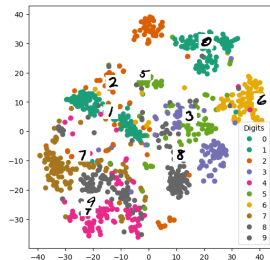
- This is worse!
- What is going on?

Hint

- We obtain better 2D representations with t-SNE: this is a hint that some “interesting” relations between pixels are non linear



(a) PCA visualization



(b) t-SNE visualization

- “Forcing” the model to explain 100% of the variance with only linear relations works on the training set, but fails to generalize

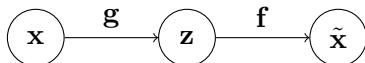
Overfitting in unsupervised learning

With too many components, we are overfitting with PCA

(more on that in a few slides)

Reconstruct simply

- We can try to map the original data $\mathbf{x} \in \mathbb{R}^D$ to a lower dimension representation $\mathbf{z} \in \mathbb{R}^H$ while still preserving a high explained variance with $\mathbf{g} : \mathbb{R}^D \rightarrow \mathbb{R}^H$
- To limit overfitting, we may use a *simple* (non linear) model
- We also need to reconstruct $\tilde{\mathbf{x}}$ back to original space from $\mathbf{z}$ with $\mathbf{f} : \mathbb{R}^H \rightarrow \mathbb{R}^D$ (we cannot do that with e.g. t-SNE)



- We want to minimize the reconstruction error:

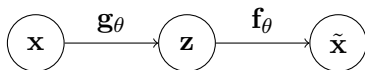
$$\underset{\mathbf{g}, \mathbf{f}}{\text{minimize}} \quad \|\mathbf{x} - \tilde{\mathbf{x}}\|_2^2 = \|\mathbf{x} - \mathbf{f}(\mathbf{g}(\mathbf{x}))\|_2^2$$

- $\mathbf{g}$ and $\mathbf{f}$ can be parametrized with $\theta_{\mathbf{g}}$ and $\theta_{\mathbf{f}}$ and written $\mathbf{g}_{\theta}$ and $\mathbf{f}_{\theta}$

Autoencoder

Definition

An *autoencoder* is a model that learns to compress data into a lower-dimensional representation (encoding) and then reconstruct it back to its original form (decoding) with the goal of minimizing reconstruction error.



$$\underset{\theta_f, \theta_g}{\text{minimize}} = \|\mathbf{x} - \mathbf{f}_\theta(\mathbf{g}_\theta(x))\|_2^2$$

- Fun fact: when both the encoder $\mathbf{g}$ and the decoder $\mathbf{f}$ are restricted to linear transformations, PCA provides the solution that minimizes the (training) reconstruction error
- Thus, overfitting with a linear auto-encoder and overfitting with PCA is equivalent

Completeness

- Assuming we learned the following 2D representation:

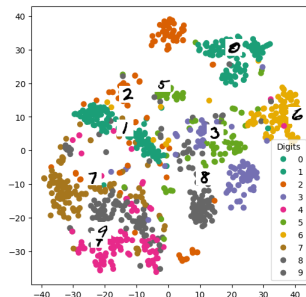


Figure: 2D latent representation of the data

- What happens if we sample between “areas” 0 and 5 in this space?
 - Corresponding reconstruction is unlikely to represent a digit

Desirable property #1

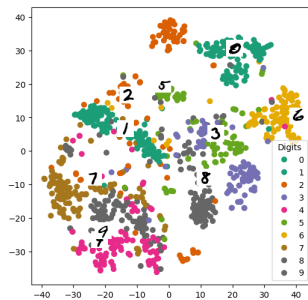
We want the learned latent space to be *complete*, i.e. to have no “gaps” leading to meaningless reconstruction of latent sampled in this area

Continuity

- Where should the following digit (left) be encoded?



(a) A 1? or maybe a 7?



(b) Latent space

- Probably somewhere between the area for 1 and the area for 7

Desirable property #2

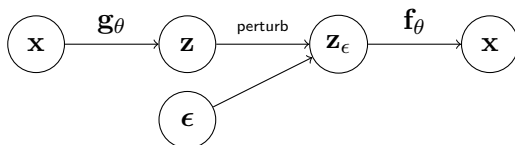
We want the learned latent space to be *continuous*, i.e. to map (and unmap) similar samples close to each other.

Achieving desirable properties

Desirable properties

We want the learned latent space to have continuity and completeness

- How can we achieve continuity?
- Idea: once we have mapped point $\mathbf{x}$ to latent point $\mathbf{z}$, we slightly move it in the latent space to $\mathbf{z}_\epsilon$ by adding some noise ϵ before reconstructing $\tilde{\mathbf{x}}$



- This forces samples close to each other in the latent space to have similar reconstructions (otherwise the reconstruction error would be very large)

Achieving desirable properties

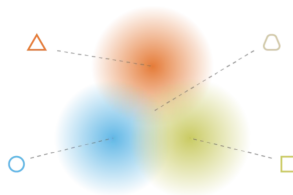
Desirable properties

We want the learned latent space to have continuity and completeness

- How can we achieve completeness?
 - ▶ We are estimating and sampling values $\mathbf{z}_\epsilon$ in latent space: we have a corresponding probability distribution $p(\mathbf{z}_\epsilon)$
 - ▶ We can force this distribution to look like a “well-behaved” distribution (e.g. a multivariate Gaussian)



(a) Nasty latent distribution



(b) Well-behaved latent distribution

Figure: from towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73

Achieving desirable properties

- How can we force the learned distribution to be similar to a Gaussian?

Achieving desirable properties

- How can we force the learned distribution to be similar to a Gaussian?
 - ▶ How can we quantify how much a (learned) probability distribution p is similar to another distribution q ?

Achieving desirable properties

- How can we force the learned distribution to be similar to a Gaussian?
 - ▶ How can we quantify how much a (learned) probability distribution p is similar to another distribution q ?
- Using Kullack-Leibler divergence $D_{\text{KL}}[q||p]$ of course!
- To prevent $p(\mathbf{z}_\epsilon) = \mathbf{g}_\theta(\mathbf{x}) + \epsilon$ from being too different from a (multivariate) normal distribution $q(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, we add a KL penalty to the objective:

$$\underset{\theta_f, \theta_g}{\text{minimize}} \sum_{n=1}^N \|\mathbf{x}_n - \hat{\mathbf{x}}_n\|^2 + \lambda D_{\text{KL}}[q||p]$$

- λ quantifies the trade-off between the reconstruction error and the similarity to a Gaussian

Summary

A *variational* auto-encoder is an auto-encoder with 2 enhancements to ensure that the learned latent space is well suited for generation:

- We add random perturbations ϵ in the latent space during training to ensure its continuity
- We regularize the reconstruction error with D_{KL} to ensure the completeness of the latent space



$$\underset{\theta_f, \theta_g}{\text{minimize}} \sum_{n=1}^N \|\mathbf{x}_n - \mathbf{f}_\theta(\mathbf{g}_\theta(\mathbf{x}_n) + \epsilon)\|^2 + \lambda D_{\text{KL}}[q||p]$$

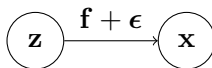
- Note: the exact formula corresponding to $D_{\text{KL}}[q||p]$ depends on the exact distribution of p and q . More on that shortly.

Sampling process

Let us forget everything we just covered (for now).

Given observable (random) variables $\mathbf{x} \in \mathbb{R}^D$, let us assume that:

- There exists a *latent distribution* with latent vectors $\mathbf{z} \in \mathbb{R}^H$
- There is a function $\mathbf{f} : \mathbb{R}^H \rightarrow \mathbb{R}^D$ mapping variables from the latent space to the observed space
- Observations are noisy: $\mathbf{x} = \mathbf{f}(\mathbf{z}) + \epsilon$



- As an example: there is a distribution over sequences of human DNA, and there exists a (pretty complex) noisy function mapping this DNA to the shape of a person's face.

³This section is partly inspired from towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73

Estimating latent distribution

- We generally only have access to observed variables $\mathbf{x}$, not to the latent variables $\mathbf{z}$
- We would like to be able to estimate $p(\mathbf{z}|\mathbf{x})$ (distribution of latent given observations), *i.e.* *encode* observed variables
- From Bayes theorem and sum rule of probability:

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{\int_{\mathcal{Z}} p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}}$$

- But even assuming that we know $\mathbf{f}$ (which is usually not the case), the integral makes it untractable in general

Variational inference

Variational inference is a generic method to approximate complex distributions:

- We select a parametrized family of distributions with parameters θ and look for the best approximation q_{θ^*} of our target distribution p among this family, so that

$$q_{\theta^*}(\mathbf{z}) \approx p(\mathbf{z})$$

- How well the approximation matches the target can be quantified by the Kullback-Leibler divergence of q_{θ} with respect to p :

$$\underset{\theta}{\text{minimize}} \ D_{\text{KL}}[q_{\theta}||p]$$

- For example, using a (multivariate) Gaussian distribution, parameters θ to estimate are the mean vector μ and covariance matrix Σ .

Variational inference for latent variable

Objective

Find $q_{\theta}(\mathbf{z}|\mathbf{x})$ with a known form based on parameters θ such that

$$q_{\theta}(\mathbf{z}|\mathbf{x}) \approx p(\mathbf{z}|\mathbf{x})$$

- Assuming that q is Gaussian, $\theta = \{\mu, \Sigma\}$

$$\begin{aligned} q_{\theta}(\mathbf{z}|\mathbf{x}) &= \mathcal{N}(\mathbf{z}|\mu_{\mathbf{x}}, \Sigma_{\mathbf{x}}) \quad (\mu \text{ and } \Sigma \text{ depend on } \mathbf{x}) \\ &= \mathcal{N}(\mathbf{z}|\mathbf{g}(\mathbf{x}), \mathbf{h}(\mathbf{x})) \end{aligned}$$

- We are looking for 2 functions $\mathbf{g} : \mathbb{R}^D \rightarrow \mathbb{R}^H$ and $\mathbf{h} : \mathbb{R}^D \rightarrow \mathbb{R}^{H \times H}$: $\mathbf{g}$ estimates the mean $\mu_{\mathbf{x}}$ of $\mathbf{z}$ given $\mathbf{x}$, and $\mathbf{h}$ its standard deviation/covariance matrix $\Sigma_{\mathbf{x}}$.
- These functions can be parametrized by θ_g and θ_h

Kullback-Leibler objective

Objective to have a q that closely matches p :

$$\underset{\theta}{\text{minimize}} \ D_{\text{KL}}[q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] = \mathbb{E}_q \left[-\log \frac{p(\mathbf{z}|\mathbf{x})}{q_{\theta}(\mathbf{z}|\mathbf{x})} \right]$$

We can show (next slides) that this is equivalent to

$$\underset{\theta}{\text{minimize}} \ \underbrace{\mathbb{E}_q[-\log p(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction error}} + \underbrace{D_{\text{KL}}[q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]}_{\text{regularization term}}$$

Kullback-Leibler objective

Proof:

$$\begin{aligned}
 D_{\text{KL}} [q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] &= \mathbb{E}_q \left[-\log \frac{p(\mathbf{z}|\mathbf{x})}{q_{\theta}(\mathbf{z}|\mathbf{x})} \right] \\
 &= \mathbb{E}_q \left[\frac{\log q_{\theta}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] \\
 &= \mathbb{E}_q [\log q_{\theta}(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q [\log p(\mathbf{z}|\mathbf{x})] \\
 &= \mathbb{E}_q [\log q_{\theta}(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q \left[\log \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} \right] \\
 &= \mathbb{E}_q [\log q_{\theta}(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q [p(\mathbf{z})] - \mathbb{E}_q [\log p(\mathbf{x}|\mathbf{z})] + \mathbb{E}_q [\log p(\mathbf{x})] \\
 &= \underbrace{\mathbb{E}_q \left[-\log \frac{p(\mathbf{x})}{q_{\theta}(\mathbf{z}|\mathbf{x})} \right]}_{=D_{\text{KL}}[q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]} + \underbrace{\mathbb{E}_q [-\log p(\mathbf{x}|\mathbf{z})] + \mathbb{E}_q [\log p(\mathbf{x})]}_{\text{constant}}
 \end{aligned}$$

Thus

$$\begin{aligned}
 &\underset{\theta}{\text{minimize}} \quad D_{\text{KL}} [q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] \\
 \iff &\underset{\theta}{\text{minimize}} \quad \mathbb{E}_q [-\log p(\mathbf{x}|\mathbf{z})] + D_{\text{KL}} [q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]
 \end{aligned}$$

VAE loss

Objective

$$\underset{\theta}{\text{minimize}} \quad \underbrace{\mathbb{E}[-\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction error}} + \underbrace{D_{\text{KL}}[q_{\theta}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]}_{\text{regularization term}}$$

To simplify things, we may further assume:

- Prior probability density function (p.d.f.) of latent $\mathbf{z}$:

$$p(\mathbf{z}) \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

(standardized multivariate Gaussian)

- Posterior p.d.f. of observation $\mathbf{x}$ given latent $\mathbf{z}$:

$$p_{\theta}(\mathbf{x}|\mathbf{z}) \sim \mathcal{N}(\mathbf{f}_{\theta}(\mathbf{z}), \sigma^2 \mathbf{I})$$

(isotropic Gaussian centered around reconstructed $\mathbf{f}_{\theta}(\mathbf{z})$)

- Posterior distribution of latent $\mathbf{z}$ given observation $\mathbf{x}$:

$$q_{\theta}(\mathbf{z}|\mathbf{x}) \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{x}}, \boldsymbol{\sigma}_{\mathbf{x}}^2 \odot \mathbf{I})$$

(Gaussian with mean vector $\boldsymbol{\mu}_{\mathbf{x}} = \mathbf{g}_{\theta}(\mathbf{x})$ and diagonal covariance matrix $\boldsymbol{\Sigma}_{\mathbf{x}} = \boldsymbol{\sigma}_{\mathbf{x}}^2 \odot \mathbf{I} = \mathbf{h}_{\theta}(\mathbf{x})$)

Reconstruction error

- We have shown⁴ for linear regression that, with the assumption $y = f_{\mathbf{w}}(\mathbf{x}) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$,

$$\begin{aligned} & \underset{\mathbf{w}}{\text{maximize}} \mathbb{E}_{\mathcal{D}}[\log p(y|\mathbf{x}, \mathbf{w})] \\ \iff & \underset{\mathbf{w}}{\text{maximize}} \frac{1}{N} \sum_{(\mathbf{x}_n, y_n) \in \mathcal{D}} \log p(y|\mathbf{x}, \mathbf{w}) \\ \iff & \underset{\mathbf{w}}{\text{minimize}} \frac{C}{N} \sum_{n=1}^N \|y_n - f_{\mathbf{w}}(\mathbf{x}_n)\|_2^2 \quad \text{for some } C \in \mathbb{R}^{+*} \end{aligned}$$

- Here, we have $\mathbf{x} = \mathbf{f}_{\theta}(\mathbf{z}) + \epsilon$, $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$, so similarly

$$\underset{\theta_f}{\text{minimize}} \mathbb{E}[-\log p(\mathbf{x}|\mathbf{z}, \theta_f)] \iff \underset{\theta_f}{\text{minimize}} \frac{C}{N} \sum_{\mathbf{x}_n \in \mathcal{D}} \|\mathbf{x}_n - \mathbf{f}_{\theta}(\mathbf{z}_n)\|_2^2$$

- i.e.* minimizing the (negative) expectation in the first term of the VAE loss is equivalent to minimizing the reconstruction error $\|\mathbf{x}_n - \hat{\mathbf{x}}_n\|_2^2$.

⁴Cf. homework assignment 1

Regularization term

Since we assumed:

$$p(\mathbf{z}) \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

and

$$q(\mathbf{z}) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2 \odot \mathbf{I})$$

$$(\text{or } q(\mathbf{z}|\mathbf{x}) \sim \mathcal{N}(\boldsymbol{\mu}_{\mathbf{x}}, \boldsymbol{\sigma}_{\mathbf{x}}^2 \odot \mathbf{I}))$$

We can show that⁵

$$D_{\text{KL}}[q||p] = \frac{1}{2} \left(\|\boldsymbol{\mu}\|^2 + \|\boldsymbol{\sigma}\|^2 - \sum_{h=1}^H \log \sigma_h^2 - H \right)$$

⁵Bonus question of homework assignment 2

Variational autoencoder loss

- So finally, our VAE objective can be written:

$$\begin{aligned}
 & \underset{\theta=\{\theta_f, \theta_g, \theta_h\}}{\text{minimize}} \\
 & \frac{1}{N} \sum_{n=1}^N \left(\underbrace{\|\mathbf{x}_n - \mathbf{f}_\theta(\mathbf{g}_\theta(\mathbf{x}_n) + \mathbf{h}_\theta(\mathbf{x}_n) \odot \boldsymbol{\epsilon}_n)\|_2^2}_{\text{reconstruction error}} \right. \\
 & \left. + \underbrace{\lambda(\|\mathbf{g}_\theta(\mathbf{x}_n)\|_2^2 + \|\mathbf{h}_\theta(\mathbf{x}_n)\|_2^2 - \sum_{h=1}^H \log(\mathbf{h}_\theta(\mathbf{x}_n))_h - H)}_{\text{KL divergence regularization}} \right)
 \end{aligned}$$

- We can then minimize this objective on the training dataset to train our VAE

Results on MNIST



Figure: Generated samples. From Vahdat & Kautz, NVAE: a deep hierarchical VAE, NeurIPS 2020

Other datasets



Figure: Generated faces from CelebA HQ dataset⁶. From Vahdat & Kautz, NVAE: a deep hierarchical VAE, NeurIPS 2020

⁶Obtaining these results requires additional tricks, such as the use of a hierarchy of VAEs. Cf. cited paper.

Latent space exploitation

- The latent space can also be used to alter the result based on specified parameters⁷

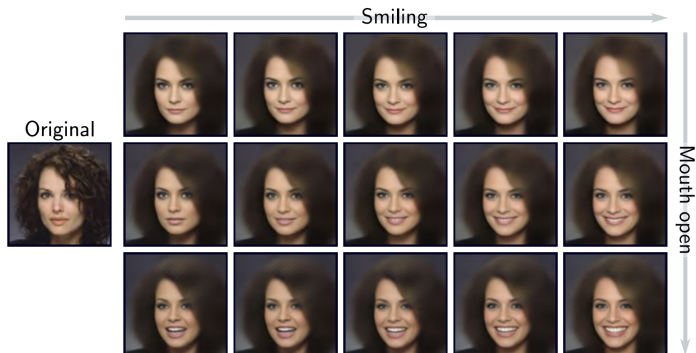


Figure: Generated faces from CelebA HQ dataset⁸. From S. Prince, Understanding Deep Learning, MIT Press 2023, adapted from T. White, Sampling generative networks, arXiv 2016

Other generative methods

- A Variational AutoEncoder is just one example of a learning algorithm tailored for generation. Other examples include, for images⁹:
 - ▶ Generative Adversarial Networks (GANs),
 - ▶ Diffusion models
 - ▶ Invertible flow networks
 - ▶ ...
- Or for text¹⁰, auto-regressive (decoder-only) transformers, used e.g. in ChatGPT

¹⁰Technically, these can be adapted for other tasks than generated images, but are mostly used for this use case. Same thing with auto-regressive transformers and text

Conclusion

- Although we typically use deep learning to do VAE, the ideas themselves are not specific to deep learning models
- This is why having good foundational knowledge of machine learning is necessary, so that we understand ideas and foundational concepts behind what we do – whether it is deep learning or not.
- Once understood, these ideas can then be applied to other tasks, adapted to other constraints, or inspire brand new ideas

Conclusion

- Although we typically use deep learning to do VAE, the ideas themselves are not specific to deep learning models
- This is why having good foundational knowledge of machine learning is necessary, so that we understand ideas and foundational concepts behind what we do – whether it is deep learning or not.
- Once understood, these ideas can then be applied to other tasks, adapted to other constraints, or inspire brand new ideas
- Thank you for your attention!